Binary Search

Por: SqueletoCosmico - Zarco - July

Club de Programación Competitiva Pu++

3 de marzo de 2026

Contenido

- Introducción
- Funciones de C++
- Busqueda binaria sobre la respuesta
- Busqueda binaria con decimales
- Minimax problems
- Búsqueda con potencias de 2

Introducción

Motivación

Supongamos que queremos determinar si un valor $x=9$ pertenece a un arreglo A de N elementos.

12	**9**	**15**	**5**	**6**	**3**	**5**	**10**	**4**	**1**	**12**	**3**

Solución Naïve

Buscar uno por uno nos lleva tiempo lineal, para Q queries sería $\mathcal{O}(QN)$.

¿Cómo lo podemos mejorar?

Preprocesamiento

¿Qué ocurre si A esta ordenado?

Podemos tomar un punto de referencia i y plantear los siguientes casos:

- Si $a_i \leq x$, entonces $a_j \leq x$ para todo $j < i$.
- Si $x \leq a_i$, entonces $x \leq a_j$ para todo $i < j$.
- Si $x = a_i$, encontramos a x.

Idea

Considerar a un rango activo $[l, r]$ donde presumimos que estaría la respuesta (de estarlo), y a partir de un punto de referencia i reducir el rango tan rápido como sea posible en el peor caso.

En el peor caso reducimos el intervalo de $[l, r]$ a $max(i - l, r - i)$, por lo que el valor óptimo de $i = \lfloor \frac{l+r}{2} \rfloor$.

Idea

1	3	3	4	5	5	6	9	10	12	12	15

↑

1	3	3	4	5	5	6	9	10	12	12	15

↑

1	3	3	4	5	5	6	9	10	12	12	15

↑

1	3	3	4	5	5	6	9	10	12	12	15

↑

Implementación

```
int l = 0, r = n-1;
while (l <= r) {
    int m = (l+r) / 2;
    if (array[m] == x) {
        return m;
    } else if (array[m] < x) {
        l = m + 1;
    } else {
        r = m - 1;
    }
 }
return -1;
```

Implementación

Cuidado

Si $l + r$ excede el rango de `int`, este calculo se puede desbordar

$$m = \frac{l + r}{2}$$

La forma segura de hacerlo sería calcular

$$m = l + \frac{r - l}{2}$$

Complejidad

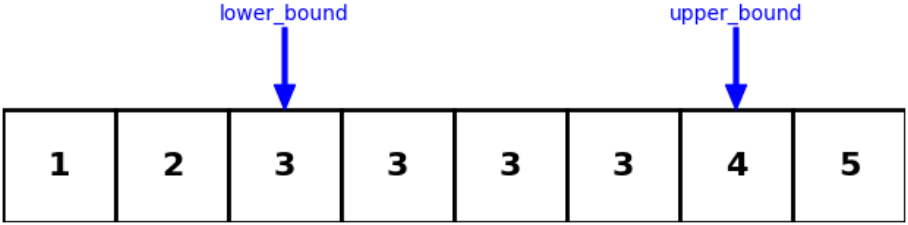
En cada iteración del algoritmo reducimos el rango activo a la mitad:

$$N \to \frac{N}{2} \to \frac{N}{4} \to \cdots \to 1$$

Obteniendo una complejidad de $\mathcal{O}(\log N)$.

Funciones de C++

Lower y Upper Bound



- `lower_bound`: Apunta al primer elemento mayor o igual a 3.
- `upper_bound`: Apunta al primer elemento mayor a 3.

STL

- `bool binary_search(first, last, val, comp)`
- `iterator lower_bound(first, last, val, comp)`
  - Si todos son mayores regresa `end()`.
  - Índice en arreglo
    `int k = lower_bound(a, a+n, x) - a`
- `iterator upper_bound(first, last, val, comp)`
  - Si todos son menores o iguales regresa `end()`
  - Cantidad de ocurrencias
    `int count = upper_bound(v.begin(), v.end(), x) - lower_bound(v.begin(), v.end(), x)`
- `pair<iterator, iterator> equal_range(first, last, val, comp)`
  - Elementos del iterador
    `*er.first, *er.second`.

Busqueda binaria sobre la respuesta

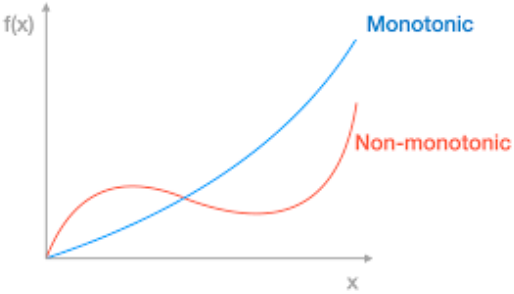
Busqueda binaria sobre la respuesta

La busqueda binaria no es solo un algoritmo de busqueda, sino que es la idea de dividir en dos nuestro espacio de busqueda, donde dicho espacio de busqueda no solo se refiere a una estructura como un arreglo ordenado, sino a cualquier función monotona.

Funciones monótonas

Función monotonamente creciente

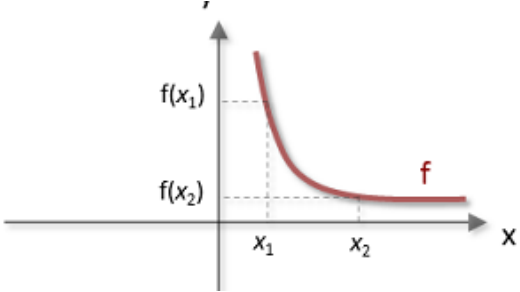
Sea f una función tal que $f : \mathbb{R} \to \mathbb{R}$. Se dice que f es monótonamente creciente si para toda $x, y \in \mathbb{R}$ tal que $x < y$ tenemos que $f(x) \leq f(y)$



Funciones monótonas

Función monotonamente decreciente

Sea f una función tal que $f : \mathbb{R} \to \mathbb{R}$. Se dice que f es monótonamente decreciente si para toda $x, y \in \mathbb{R}$ tal que $x < y$ tenemos que $f(x) \geq f(y)$



Busqueda binaria sobre la respuesta

Si tenemos un dominio o intervalo de posibles valores en los que se puede encontrar nuestra respuesta, entonces podemos hacer busqueda binaria para encontrarla, pero solo si nuestro espacio de busqueda tiene la forma de una función monótona.

Cada vez que saquemos la mitad de nuestro intervalo, digamos que es el valor m, vamos a pobrar si dicha posible respuesta m soluciona nuestro problema, y dependiendo de si lo hace o no, reducimos nuestro espacio de busqueda a la derecha o a la izquierda.

Problema de práctica

- **Books**

  Judge: Codeforces

Easy

Busqueda binaria con decimales

Busqueda binaria con decimales

Si tenemos $l = 0{,}0$ y $r = 10{,}0$, donde $l, r \in \mathbb{R}$, habrá un número infinito de numeros reales en el intervalo $[l, r]$. Pero en números decimales con dos cifras después del punto hay...

Busqueda binaria con decimales

En este caso tenemos una presición de 10^{-2}, entonces si bien hay 10 enteros entre 1 y 10, si contamos la presición tendremos un total de 1000 números:

$$
\begin{aligned}
1{,}00 \\
1{,}01 \\
\vdots \\
2{,}00 \\
2{,}01 \\
\vdots \\
10{,}00
\end{aligned}
$$

En general, podemos decir que si buscamos presición de ϵ, entonces hace falta considerar $\frac{r-l}{\epsilon}$ números

Búsqueda binaria con decimales

Comúnmente hay problemas donde nos dicen que la respuesta será considerada si el error absoluto no es más grande que 10^{-6}. Entonces, si por ejemplo tenemos:

$$l = 0{,}0$$
$$r = 10^9$$
$$\epsilon = 10^{-6}$$

Consideraremos un total de:

$$\frac{r - l}{\epsilon} = \frac{10^9 - 0}{10^{-6}} = 10^9 \cdot 10^6 = 10^{15}$$

¿Cómo hacemos binary search con decimales?

Sabemos que el algoritmo de binary search divide el espacio de busqueda en 2 cada vez, y con lo anterior vimos que nuestro espacio de busqueda es de $\frac{r-l}{\epsilon}$ números, entonces binary search tomará $\log_2(\frac{r-l}{\epsilon})$ pasos en llegar a la respuesta.

Entonces en vez de hacer un `while`$(l \leq r)\ldots$, es mejor hacer un número de iteraciones correspondinete al valor de $\log_2(\frac{r-l}{\epsilon})$

Problema ejemplo

Equation

Encuentra el número x tal que $x^2 + \sqrt{x} = c$.

Input:

Solo un número $c \in \mathbb{R}$ tal que $1{,}0 \leq c \leq 10^{10}$

Output:

Imprime el número x requerido. La respuesta será considerada correcta si el error absoluto no es más grande que 10^{-6}

Ejemplo

input: 15,6

output: 3,698232168829691

Solución a ’Equation’

```
#include <bits/stdc++.h>
using namespace std;

int main() {

  double c;
  cin >> c;

  double l = 0.0;
  double r = 1e5;
  for (int i = 0; i < 40; i++) {
    double mid = l + (r - l) / 2;
    double equation = mid * mid + sqrt(mid);

    if (equation > c) {
      r = mid;
    } else {
      l = mid;
    }
  }

  cout << setprecision(10) << l << endl;
  return 0;
}
```

Consejo

Observación: Se hace $l = mid$ y $r = mid$ y no $l = mid \pm 1$ o $r = mid \pm 1$ pues eso nos alejaría más de la respuesta

Consejo

Nota: En general, con un número de iteraciones de 60 para el `for` es suficiente para muchos problemas, pero se acostumbra a poner 100 o 200

Minimax problems

Minimax

Son problemas donde necesitas minimizar un valor que es el máximo de otros valores: $\text{mín}(\text{máx}(\ldots))$. Este tipo de problemas podemos reducirlos a **binary search**.

Minimax ejemplo

Get together

Tenemos a n personas en una línea recta que necesitan reunirse en un punto. Cada persona tiene su posición x_i y su velocidad v_i. Debes encontrar el mínimo tiempo en el que se pueden reunir.

Input:

n ($1 \leq n \leq 10^5$), seguido de n líneas con x_i y v_i ($-10^9 \leq x_i \leq 10^9, 1 \leq v_i \leq 10^9$).

Output:

El mínimo tiempo con un error no mayor a 10^{-6}.

Minimax ejemplo

Get together ejemplo

Input:

5
-1 5
10 3
4 2
7 10
8 1

Output:

1,5

Explicación

¿Cuánto se van a tardar estas personas en llegar a este punto? Esto va a depender de que tan rápido caminen. Dado que cada persona i tiene una velocidad v_i, entonces cada persona llegará al punto x en tiempo $\frac{|x_i - x|}{v_i}$

Y el tiempo en el que todas las personas llegan al punto x será entonces el máximo de ese valor para todas las personas

Cómo queremos minimizar lo anterior, necesitamos encontrar x en la expresión $\text{mín}_x(\text{máx}_i \frac{|x_i - x|}{v_i})$

Explicación

En vez de minimar el valor $\text{máx}_i(\frac{|x_i - x|}{v_i})$, podemos acotarlo por un valor t (tiempo).

Entonces haremos una función $f(t)$ que revise si $\text{máx}_i(\frac{|x_i - x}{v_i}) \leq t$. Dicha función es monótona, pues si t cumple la condición, entonces cualquier valor más grande que t también la cumple: $\text{máx}_i(\frac{|x_i - x}{v_i}) \leq t \leq t'$

Si encontramos el mínimo valor para el cual $f(t) = 1$, es decir, la función f se cumple, entonces t será la respuesta al problema.

Explicación

Podemos buscar t pero, ¿qué pasa con x? También es una variable. Para entender mejor el problema, conviene reescribirlo:

$$
\begin{aligned}
\max_i \left(\frac{|x_i - x|}{v_i} \right) \leq t \iff & \frac{|x_i - x|}{v_i} \leq t, && \forall i \\
\iff & |x_i - x| \leq t \cdot v_i, && \forall i \\
\iff & |x - x_i| \leq t \cdot v_i, && \forall i \\
\iff & -t \cdot v_i \leq x - x_i \leq t \cdot v_i, && \forall i \\
\iff & -t \cdot v_i + x_i \leq x \leq t \cdot v_i + x_i, && \forall i \\
\iff & x \in [x_i - t \cdot v_i, x_i + t \cdot v_i], && \forall i
\end{aligned}
$$

Realmente no necesitamos encontrar a la x si no nos la piden, basta con ver que **exista**, y va a existir una x en todos esos intervalos si la intersección de los mismos es **no vacía**

Intersección intervalos

Supongamos que tenemos n segmentos: $(a_0, b_0), (a_1, b1), \ldots, (a_n, b_n)$.

Para encontrar la intersección de todos basta con encontrar el máximo de todos los limites izquierdos, i.e, $\text{máx}_i(a_i) = L$, y el mínimo de todas los límites derechos, i.e, $\text{mín}_i(b_i) = R$.

Si pasa que $R < L$, eso quiere decir que no hay intersección entre todos los segmentos. En otro caso, la intersección es $[L, R]$

Solución

Entonces ya solo basta con hacer busqueda binaria sobre t, y si $f(t) = 1$ (esto es, la intersección de los segmentos para esa t es no vacía) entonces guardamos a t como respuesta y nos vamos a la izquierda (pues queremos minimizar el tiempo) y en otro caso nos vamos a la derecha.

Al final, la última t que cumplió la condición será el valor mínimo del tiempo que buscamos.

Problema de práctica

- **Get together**

  Judge: Codeforces

Practice

Búsqueda con potencias de 2

Idea

Otra manera de hacer búsqueda binaria es mediante saltos. Si comenzamos en la posición `pos=0` y en cada iteración avanzamos mientras `p+jump` no salga del arreglo y `A[p+jump]` no exceda el valor buscado x.

Realizando saltos desde $\frac{N}{2} \to \frac{N}{4} \to \cdots \to 1$, *i.e.*, disminuyendo la velocidad conforme nos acercamos al valor buscado.

Obtenemos una complejidad $\mathcal{O}(\log N)$, ya que cada salto puede realizarse a lo más dos veces.

Saltos

1	3	3	4	5	5	6	9	10	12	12	15

1	3	3	4	5	5	6	9	10	12	12	15

1	3	3	4	5	5	6	9	10	12	12	15

1	3	3	4	5	5	6	9	10	12	12	15

Implementación

```
int pos = 0;
for (int jump = n/2; jump>=1; jump/=2){
    while(pos+jump < n && array[pos+jump] <= x){
        pos += jump;
    }
 }
if (array[pos]==x){
    // x found at index pos
 }
```